

HW2 任务一：前缀缓存测量结果

实验设置

SGLang 0.5.14 (CPU, torch_native), Qwen/Qwen3-0.6B, sampling_seed=2026, temperature=0, max_new_tokens=16, ignore_eos=true, RadixCache 开启, 原生 /generate + input_ids + 流式响应; 每组 32 条, 最大并发 8, 输入 512 tokens = 256 共享前缀 + 256 独有后缀; 预热 + flush_cache 后测量, shared 组先发一条预热请求把共享前缀写入缓存(不计入结果)。

对照表

两组对照表

指标	dispersed_prefix	shared_prefix
成功率	32/32	32/32
吞吐 (req/s)	0.1892	0.2461
缓存命中 tokens	0	8192
实际 Prefill tokens	16384	8192
TTFT p50/p95 (s)	26.464/27.46	17.11/17.31
TPOT p50/p95 (s)	1.074/1.721	1.0318/1.09
E2E p50/p95 (s)	42.208/42.574	32.498/32.612
缓存命中率	0.0	0.5

结果分析（因果自注意力公式视角）

说明

$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k}) \cdot V$ 中位置 i 的输出只依赖 K/V 的前 i 个位置。
因此相同 token + 相同位置的前缀, 无论出现在哪个请求里, 计算出的 K/V 完全相同 —— 这就是前缀可复用的数学依据。
命中后仍需计算: 共享前缀之后的全部 token (本实验为 256 个独有后缀 token 的 $Q/K/V$ 及其输出)。
TTFT 变化: shared 组 Prefill 从 512 降到 256 tokens, p50 TTFT 26.46s -> 17.11s (-35%)。
TPOT 变化: decode 每步只算 1 个新 token 的 $Q/K/V$, 成本与缓存命中无关, 故两组 TPOT p50 基本一致 (1.074 vs 1.032s), 仅因 batch 状态

HW2 任务二: /generate 请求源码流程图与说明

流程图 (文本版)

POST /generate

```
-> TokenizerManager.generate_request [managers/tokenizer_manager.py]
-> tokenize -> Req -> ZMQ -> Scheduler
-> event_loop_normal / waiting_queue [managers/scheduler.py]
-> get_new_batch_prefill [managers/scheduler.py]
-> match_prefix_for_req -> RadixCache.match_prefix [mem_cache/radix_cache.py]
    (命中前缀 -> 复用 KV; 未命中 -> 需 Prefill)
-> PrefillAdder [managers/schedule_policy.py]
    / ScheduleBatch [managers/schedule_batch.py]
-> TpModelWorker: Prefill(首token) -> Decode(自回归) [managers/tp_worker.py]
-> cache_finished_req / cache_unfinished_req [mem_cache/radix_cache.py]
-> DetokenizerManager -> TokenizerManager streaming output -> Client
```

说明与问答

SGLang v0.5.14 /generate 请求主流程追踪

流程总览 (对应流程图)

1. **请求接收与排队**: FastAPI 收到 POST /generate, `TokenizerManager.generate\_request()` (python/sglang/srt/entrypoints/http_server.py 调用; 实现在 python/sglang/srt/managers/tokenizer_manager.py) 把文本/ input_ids tokenize 成 Req, 经 ZMQ 发给 Scheduler, 进入 `schedule\_r.waiting\_queue`。
2. **调度与前缀匹配**: Scheduler 事件循环 `event\_loop\_normal()` (python/sglang/srt/managers/scheduler.py) 每轮调用 `get\_new\_batch\_prefill()` (实现在 managers/schedule_r.py, 内部使用 managers/schedule_policy.py 的 PrefillAdder 做预算控制), 对每个 Req 执行 `match\_prefix\_for\_req()` -> `RadixCache.match\_prefix()` (python/sglang/srt/mem_cache/radix_cache.py), 沿 radix 树按 token 前缀匹配已缓存的 KV, 命中部分标记 prefix_indices, 未命中部分才需要 Prefill。
3. **Prefill**: `ScheduleBatch` (managers/schedule_batch.py) 合并后发给 `TpModelWorker` (managers/tp_worker.py), 对未命中 token 做一次前向计算, 产出首个输出 token 与新 KV。
4. **Decode**: 请求进入 running batch, `event\_loop\_normal` 每步对整批做一次 decode 前向, 自回归生成直到 EOS 或 max_new_tokens。
5. **缓存写回**: 请求完成后 `RadixCache.cache\_finished\_req()` (未完成批次用 `cache\_unfinished\_req()`) 把新产生的 KV 按 token 前缀插入 radix 树, 供后续请求复用; LRU 淘汰。
6. **流式返回**: TokenizerManager 的 stream_out 循环把每个新 token 封装为 JSON 行 (data: {...}) 经 HTTP chunked 推回客

户端; `meta\_info.finish\_reason` 终止。

关键问答

- **如何进入等待队列**：TokenizerManager token 化后经 ZMQ socket 发送，Scheduler 主循环 recv_requests() 后 append 到 waiting_queue，按 fcfs/lpm 等策略被 PrefillAdder 取出组批。
- **前缀匹配如何减少 Prefill token**：RadixCache 以 token 序列建基数树，match_prefix 返回最长可复用前缀的 KV 索引；请求只需对后缀做 Prefill（本次实验 shared 组 512 token 中 256 命中，Prefill 减半）。
- **新 KV 如何写回**：Prefill/Decode 产出的 KV 位于 token pool；请求结束（或每次 chunk）时 cache_finished_req/cache_unfinished_req 以 token 序列为键把 KV 页插入 radix 树，引用计数管理生命周期。
- **流式返回**：DetokenizerManager 增量反解码，TokenizerManager 按生成顺序逐 token 通过 asyncio queue 写回 HTTP 流式响应。

HW2 AI 使用说明情况

使用的模型

Hermes Agent 大模型代理 (GLM/Claude 级)。

提示词

任务书 PDF 附件 + "完成这两个项目并进行核验"、"进度如何" 等进度查询指令。

AI 帮助

解读任务书与 SGLang v0.5.14 源码结构定位关键函数; 编写两组负载测量脚本; 排查流式响应 data: 前缀解析 bug; 汇总 p50/p95 统计。

是否被误导

第一版流式解析误按纯 JSON 行处理导致全部 timeout, 属于 AI 生成的解析逻辑与 SGLang 实际 SSE 格式不符, 通过 curl 抓原始流修正。

HW2-5 作业感受（第二次挑战）

（1）如何完成第二次挑战

主体流程和第一次类似：先把 SGLang 0.5.14 的 CPU 服务跑起来，再按任务书把测量脚本写好。AI 代理负责搭环境、写脚本初稿和读源码定位函数，我负责对照任务书核对硬性参数（每组 32 条、并发 8、`sampling_seed=2026`、`ignore_eos`、`max_new_tokens=16` 这些一个都不能错），每组跑完就打开 `per_request.jsonl` 抽查几条，看 `input/output tokens`、`cached_tokens` 对不对得上，不对就回去查脚本而不是先怀疑机器。迭代下来最大的体会是验收不能偷懒，有一轮脚本残留了 32 条 `timeout` 记录在 `jsonl` 里，就是逐行翻文件才发现的。

（2）最困难的部分

最折腾的是流式解析的 bug。第一版脚本把响应体当纯 JSON 行去解析，结果 32 条请求全部 `timeout`，一开始还以为是 CPU 推理太慢扛不住并发。后来用 `curl` 直接抓原始响应，看到每行前面带着 `data:` 前缀，才知道这是 SSE 格式，解析层先剥掉前缀就通了。这件事给我的教训是：调网络接口的问题，第一步永远是先看原始字节，别急着改业务逻辑。另一个困难是 CPU 上跑 512 token 的 `prefill` 要二十多秒，一组实验两三分钟，试错成本很高，逼着我在每次重跑前把参数全部检查一遍再发请求。

（3）对科研的启发

第一，先跑通最小闭环再谈优化，一个 32 条请求的小实验跑通了，比一个设计了很久但没跑起来的框架有价值。第二，数据要能溯源：报告里每个数字都应该能对到原始 `jsonl` 里的某一行，这次对照表就是从逐请求记录重新算出来的，心里很踏实。第三，不能全信 AI：这次它把 `get_new_batch_prefill` 的位置标到了 `schedule_policy.py`，我翻 v0.5.14 源码才发现函数其实在 `scheduler.py` 里，AI 给的结论必须自己验证过才能写进报告。第四，资源受限时找替代路径也是一种能力，没有 GPU 就用 CPU 加 `torch_native`，速度慢但对照组的结论方向照样成立。